The architecture of a self-hosted coding agent

How DiveeOI is put together: one primary server process, subsystem-owned data stores, a layered service graph, and services — sessions, tools, providers, background tasks — that share the application context.

1 · The deployment shape

DiveeOI is a Bun process that serves both an HTTP API and the built single-page client. The browser is a view; the server owns sessions, tools, permissions and the core app data. Optional supervised child processes (for example, the persistent Rust retrieval engine) do not require a separate always-on service or message broker.

Core application state lives in SQLite under the application data directory; the legacy `OPENCODE_DB` override remains supported. Targeted Learning deliberately owns separate `books.db` (books, passages and vectors) and `student.db` (student memory and notes) stores under its own data directory. Schema changes in the core database are migrations applied on boot. Bun is the primary runtime; Node 22 is supported for the SQLite bridge path.

2 · The package map

PACKAGE	RESPONSIBILITY	SCALE
server	HTTP surface, sessions, tools, providers, the task subsystem	Effect service graph and runtime orchestration
db	schema, migrations, provider adapters, the session engine	Core SQLite persistence
api	HttpApi group and schema definitions	Typed endpoint and OpenAPI contract
app / ui	SolidJS client and shared component library	Vite, themes, reusable components
learning	Targeted Learning subsystem	Ingestion/OCR, BGE-M3, hybrid retrieval, student memory, trainer
llm	the schema-first provider boundary	model protocol and routing
memory	the project memory graph	extraction, recall, consolidation
plugin, plugins	the plugin SDK and the bundled plugins	hooks, tools, config
mcp, identity, sdk	the MCP host, the brand assets, the clients	published separately

3 · The request path

```
browser ── fetch /session/:id/message
  → HttpApi (packages/api)            →  session context + permissions
  → SessionPrompt                     →  the turn loop
  → ToolRegistry                      →  the tools this agent may use
  → tool (read / grep / edit / bash)  →  each behind a permission rule
  → Provider                          →  the model, resolved for this turn
  → events                            →  streamed back over SSE
  → SessionProcessor                  →  parts persisted
  → SQLite
```

The provider boundary is the interesting part. A model id is looked up in the catalogue at the moment it is needed, not at boot, so a provider can be added, removed or refused while the server runs. DiveeComposer — the synthetic model the user picks — resolves to a concrete `provider/model` per turn and carries that model's capabilities, so the client's attachment affordances describe the model that will actually answer.

4 · The service graph

Every service is an Effect layer. `LayerNode.make(layer, deps)` declares one and the dependency checker verifies the declaration at compile time; `buildLayer(graph)` assembles a graph, with a cycle check at assembly.

The HTTP application graph is a single group. The routes, the task runner and the task generator all draw their services from it. This matters more than it looks: a service that reads another service *at call time* pushes that requirement onto whoever calls it, so a worker that does not share the application context fails at runtime with a service-not-found error, however complete its own dependency list is. Sharing one list removes that class of failure.

5 · The data model

Twenty-six tables. The interesting ones are the two shapes of a conversation: a v1 shape (`session_message`, `session_input`) and a v2 event shape (`session_message`, `session_context_epoch`), where the epoch row holds the context snapshot a session resumes from. Around them: `todo`, `reminder`, `preference`, `permission`, `credential`, `project`, `session_anchor` for the lossless memory, and the three task tables.

6 · The background task subsystem

A task is a session with a schedule, plus a thin overlay of state: status, heartbeat, run count, next run, hook definitions, and provenance — who made it, and which agent.

- **Two queues.** The runner polls tasks whose cron is due and tasks left `queued`. Capacity is adaptive, so a slow machine runs fewer tasks at once.
- **One prompt path.** A task runs through the same loop an interactive session uses, in the same instance context, with the same permissions.
- **A call journal.** Each tool call is written before it executes and marked committed after. A crash replays committed calls and surfaces the interrupted one as a prompt to verify — so a resume never repeats a side effect.
- **Hooks.** Triggers (`on_complete`, `on_fail`, `on_timeout`, `on_output_match`) map to actions (run a command, notify, webhook, create a task). Every dispatch is written to an audit trail.
- **A generator.** A task can be handed a one-line request; the generator agent writes the full spec — prompt, schedule, agent, models — and stops. The task's own agent then runs it, unattended.
- **Agent-facing tools.** `task_create`, `task_list`, `task_status`, `task_logs`, `task_control`, so an agent can manage scheduled work from inside a session and record its own provenance.

7 · The model layer

Providers are adapters behind one interface. A quality classifier scores candidates on family, version lineage, variant words and catalogue capabilities; a reachability preference favours the bundled free tier, which needs no key; live evidence then dominates the prior, because a model that answered minutes ago is worth more than a model that merely looks capable. A model that errors, or answers with nothing, is benched for a day, and the bench is persisted.

8 · The extension surface

Agents, commands, skills, plugins, MCP servers and permissions are all files or configuration, discovered at runtime: `.opencode/agent/*.md`, `.opencode/command/*.md`, `SKILL.md` folders, plugin modules, and MCP server declarations. Permissions are evaluated per tool and per pattern, last match wins, and a deny is a deny.

9 · The media pipeline

The two films are part of the product's surface, and they are built the same way the app is: the brand, the palette and the motion language live in two documents, and everything visual is derived from them rather than re-decided per scene.

Each film is a HyperFrames project whose root composition holds only timed hosts; every scene is its own sub-composition, mounted with `data-composition-src`. That is what makes the timeline in the editor show one row per scene instead of a wall of nested divs. A scene's code speaks in absolute film time — "the diff lands at 21.0s" — and the build shifts it into the scene's own slot, so a scene file never has to know where it sits.

The UI in the films is a recreation of the real screens, taken from the source: the same theme tokens, the same geometry, the same copy. Two rules keep the renders reproducible — one paused timeline per scene, and no clocks or randomness, so a frame is a pure function of t.

10 · Testing

The suite is layered the way the code is: unit tests beside the module they cover, integration tests against a real server and a real database, and the app's own tests against the DOM. Fixtures create a temporary project directory, bind it as the active instance, and dispose it afterwards, so a test never touches a real workspace.

Where a test needs to own a service — a mocked HTTP client, a particular config, a fixed clock — it provides that service itself rather than inheriting one. The ambient fixture exists for the opposite case: a graph assembled by hand has to supply every service its layers transitively require, and a missing one shows up as a service-not-found error during construction, which takes the whole file down before the first assertion.

11 · Security posture

- **Authentication modes.** Basic authentication and JWT/account modes are supported. Production requires explicit strong secrets; registration is closed after the first account by default.
- **Permissions are the real boundary.** Every tool call is matched against the ruleset — per tool, per pattern, last match wins — and a deny is a deny. The agent cannot widen its own permissions.
- **External paths are gated.** A tool that reaches outside the project asks first, and a rule can refuse a whole class of paths.
- **External data paths are explicit.** Provider keys are held by the self-hosted instance, but prompts go to the provider you choose; web tools and remote MCP servers also receive the requests you send them.
- **The network surface is what you expose.** The server binds what you tell it to; behind a proxy, terminate TLS there. The database file is as sensitive as the code in it.

12 · Known limits

- There is no terminal client; the product is browser-first.
- There is no built-in TLS terminator; put the server behind HTTPS at a reverse proxy.
- SQLite serialises writers; this is a self-hosted, single-node architecture rather than a horizontally scaled database service.
- Provider, web-search and remote MCP integrations are network boundaries; configure and review them deliberately.
- The bundled free tier is free and below a frontier model on hard reasoning.

13 · Targeted Learning and Retrieval V2

Targeted Learning is a separate subsystem for book-grounded sessions. It extracts text from PDF, DOCX and text sources, performs OCR where needed, chunks passages and indexes them with the pinned BGE-M3 teacher. A session selects books and retrieves evidence before the model is called; a relevance gate and citation verifier keep the answer tied to the passages shown for that turn.

Its storage is deliberately separate from ordinary project sessions: `books.db` holds the library, chunks, vectors and ingest state; `student.db` holds student memories and book notes. Both live under the configurable Targeted Learning data directory.

- **Hybrid retrieval.** SQLite FTS5/BM25 and dense cosine candidates are fused with RRF; an optional persistent Rust engine performs the hot path and TypeScript remains the fallback.
- **Teacher and student.** BGE-M3 indexes passages. A compact query student can replace it for chat only after held-out evaluation writes a promoted runtime manifest; the vector dimension and pooling contract stay compatible with existing indexes.
- **Policy boundary.** Query rewriting, relevance gates, parent expansion, session integration and grounding verification remain in TypeScript.
- **Offline tooling.** Dataset generation, multilingual evaluation, ONNX export, corpus preparation and promotion tooling live outside the normal server process.